

Fundamentos del análisis numérico

Representación, errores,
condicionamiento y estabilidad

Oswaldo José Velásquez Castañón
Métodos Numéricos – 2026-2



UNIVERSIDAD
DEL PACÍFICO

El orden de cálculo cambia el resultado

En aritmética exacta

$$(a + b) + c = a + (b + c).$$

En punto flotante

Para $a = 10^{16}$, $b = -10^{16}$ y $c = 1$:

$$(a + b) + c = 1, \quad a + (b + c) = 0.$$

La forma de evaluar una expresión también forma parte del método numérico.

Objetivo: auditar un resultado numérico

- ▶ Reconocer qué números puede representar una computadora.
- ▶ Distinguir errores de entrada, truncamiento y redondeo.
- ▶ Medir errores absolutos, relativos, hacia adelante y hacia atrás.
- ▶ Aplicar análisis diferencial para estimar cómo se propagan perturbaciones.
- ▶ Identificar cancelación, desbordamiento y pérdida de precisión.
- ▶ Comparar esquemas de evaluación de un mismo problema.
- ▶ Separar el **condicionamiento del problema** de la **estabilidad del algoritmo**.
- ▶ Comparar exactitud y costo mediante experimentos con NumPy.

Tres errores exigen respuestas distintas

Entrada

Los datos ya llegan perturbados por medición, registro o redondeo previo.

Truncamiento

Se reemplaza un proceso infinito o continuo por uno finito y computable.

Redondeo

Cada resultado se aproxima por un número representable en la máquina.

Identificar la fuente del error determina cómo podemos controlarlo.

El error entra en varias etapas

Datos

Medición, muestreo, redondeo previo o registro incompleto.

Modelo

Simplificaciones que reemplazan el fenómeno por un problema tratable.

Cálculo

Discretización, terminación de iteraciones y aritmética finita.

Dato real $\longrightarrow$ modelo $\longrightarrow$ algoritmo $\longrightarrow$ resultado de máquina

El péndulo distingue los errores

El modelo ideal satisface

$$\theta''(t) + \frac{g}{L} \sin \theta(t) = 0.$$

Para ángulos pequeños se usa $\sin \theta \approx \theta$ y se resuelve

$$\theta''(t) + \frac{g}{L} \theta(t) = 0.$$

- ▶ La sustitución de $\sin \theta$ produce **error de modelo**.
- ▶ Aproximar los datos iniciales produce **error de entrada**.
- ▶ Resolver la ecuación en una computadora produce **error numérico**.

Truncar reemplaza un problema por otro

La expansión

$$e = \sum_{j=0}^{\infty} \frac{1}{j!}$$

no puede evaluarse con infinitos términos. Usamos

$$e_N = \sum_{j=0}^N \frac{1}{j!}.$$

Error de truncamiento

$$e - e_N = \sum_{j=N+1}^{\infty} \frac{1}{j!}.$$

Este error existe incluso si todas las operaciones se realizaran exactamente.

Una derivada muestra dos errores diferentes

Para aproximar $f'(x)$ usamos una diferencia hacia adelante:

$$D_h f(x) = \frac{f(x+h) - f(x)}{h}.$$

- ▶ Si h es grande, domina el **error de truncamiento**: la secante aproxima pobremente a la tangente.
- ▶ Si h es extremadamente pequeño, la resta $f(x+h) - f(x)$ pierde cifras y domina el **error de redondeo**.

Consecuencia

Reducir el tamaño de paso no mejora indefinidamente una aproximación numérica.

La base determina los dígitos disponibles

En base β , un número tiene una expansión

$$x = \pm \sum_{j=-\infty}^m d_j \beta^j, \quad d_j \in \{0, 1, \dots, \beta - 1\}.$$

Por ejemplo,

$$18 = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2 + 0 = 10010_2.$$

- ▶ Las computadoras habituales emplean representación binaria.
- ▶ Una expansión decimal finita puede ser infinita en base 2.

Punto fijo y flotante reparten el espacio

Punto fijo

Se reservan posiciones fijas antes y después del separador.

- ▶ separación uniforme;
- ▶ rango limitado.

Punto flotante

Una mantisa se combina con un exponente.

- ▶ rango mucho mayor;
- ▶ separación dependiente de la escala.

El análisis numérico moderno trabaja principalmente con punto flotante.

La máquina tiene cuatro parámetros

Un sistema idealizado puede describirse por

$$\mathcal{F}(\beta, t, L, U) = \{\pm(0.d_1 \cdots d_t)_\beta \beta^e : d_1 \neq 0, L \leq e \leq U\} \cup \{0\}.$$

- ▶ β fija la base y t la cantidad de dígitos significativos.
- ▶ L y U fijan los exponentes mínimo y máximo.
- ▶ El conjunto es finito y su separación no es uniforme en la recta real.

Los números de máquina son finitos

En base β , un número normalizado tiene la forma

$$x = \pm(0.d_1d_2\cdots d_t)_\beta \beta^e, \quad d_1 \neq 0.$$

- ▶ t controla la precisión de la mantisa.
- ▶ El rango permitido de e controla los números más pequeños y más grandes.
- ▶ La mayoría de los números reales no se representa exactamente.

Modelo de redondeo

Si no hay desbordamiento y x está en el rango normal,

$$\text{fl}(x) = x(1 + \delta), \quad |\delta| \leq u.$$

Redondear elige un número de máquina

Si $x \notin \mathcal{F}$, la operación $\text{rd}(x)$ selecciona un número representable. Con redondeo al más cercano,

$$|x - \text{rd}(x)| \leq |x - g| \quad \text{para todo } g \in \mathcal{F}.$$

En el rango normal podemos escribir

$$\text{rd}(x) = x(1 + \delta), \quad |\delta| \leq u.$$

Lectura

La cota es relativa: el error permitido se adapta a la magnitud de x .

Las operaciones también se redondean

Para $x, y \in \mathcal{F}$ definimos

$$x \oplus y = \text{rd}(x + y), \quad x \ominus y = \text{rd}(x - y),$$

$$x \otimes y = \text{rd}(xy), \quad x \oslash y = \text{rd}(x/y).$$

Cuando no hay excepciones,

$$x \circledast y = (x * y)(1 + \delta), \quad |\delta| \leq u.$$

Por ello, las identidades algebraicas reales no siempre se conservan en máquina.

float64: rango amplio, precisión finita

Representación binaria64

- ▶ 1 bit de signo;
- ▶ 11 bits de exponente;
- ▶ 52 bits explícitos de fracción.

Dos escalas distintas

$$\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16},$$

$$u = \frac{\epsilon_{\text{mach}}}{2} = 2^{-53}.$$

Fuente: documentación oficial de `numpy.finfo`.

La notación $\text{fl}(E)$ fija el esquema

Para una expresión E , $\text{fl}(E)$ denota el resultado obtenido al evaluarla en aritmética de punto flotante.

$$\text{fl}(a + (b + c)) = a \oplus (b \oplus c),$$

$$\text{fl}((a + b) + c) = (a \oplus b) \oplus c.$$

Punto central

La expresión matemática y el orden concreto de sus operaciones son objetos distintos para el análisis del error.

NumPy inspecciona la aritmética

Experimento

```
info = np.finfo(np.float64)
print(info.eps)
print(info.smallest_normal)
print(info.smallest_subnormal)
print(info.max)
```

- ▶ `np.spacing(x)` mide la separación local entre números.
- ▶ `np.nextafter(x, y)` entrega el siguiente representable desde x hacia y .

Fuente: <https://numpy.org/doc/stable/reference/generated/numpy.finfo.html>

0.1 no tiene expansión binaria finita

Así como $1/3 = 0.333\dots$ en base 10, el número $1/10$ tiene una expansión infinita en base 2. Por ello se almacena el número binario64 más cercano.

Consecuencia visible

$$0.1 + 0.2 = 0.30000000000000004$$

en la representación usual de Python.

- ▶ No es un error de Python.
- ▶ Es una consecuencia de representar números reales con bits finitos.

La suma no es asociativa en punto flotante

Para $a = 10^{16}$, $b = -10^{16}$ y $c = 1$:

$$\text{fl}(\text{fl}(a + b) + c) = 1,$$

$$\text{fl}(a + \text{fl}(b + c)) = 0.$$

Al sumar 1 a -10^{16} , el incremento es demasiado pequeño para modificar el número almacenado.

Idea clave

Operaciones equivalentes en números reales pueden inducir algoritmos distintos.

Rango y precisión son limitaciones diferentes

Overflow

El resultado supera el máximo representable y puede convertirse en ∞ .

Underflow

El resultado se aproxima a cero; aparecen números subnormales y menor precisión.

Separación

La distancia entre números representables aumenta con su magnitud.

```
np.spacing(1.0)  $\neq$  np.spacing(1e16)
```

Error absoluto y error relativo

Sea x el valor exacto y $\hat{x}$ una aproximación.

$$E_{\text{abs}} = |x - \hat{x}|, \quad E_{\text{rel}} = \frac{|x - \hat{x}|}{|x|}, \quad x \neq 0.$$

- ▶ El error absoluto conserva las unidades del problema.
- ▶ El error relativo compara el error con la escala de x .
- ▶ Cerca de cero, el error relativo puede dejar de ser informativo.

Ejemplo

Un error de $S/1$ puede ser pequeño en un presupuesto de $S/1$ millón y enorme en un saldo de $S/2$.

Cifras correctas y error relativo

Si

$$E_{\text{rel}} < 5 \times 10^{-m},$$

se suele interpretar que la aproximación posee al menos m cifras significativas correctas bajo el criterio decimal adoptado.

No confundir

- ▶ **Cifras significativas:** dependen de la escala del número.
- ▶ **Cifras decimales:** se cuentan desde el punto decimal.

Reportar muchos dígitos no implica que todos sean confiables.

Errores hacia adelante y hacia atrás

Para un problema $y = f(x)$ y un resultado calculado $\hat{y}$:

- ▶ **Error hacia adelante**: distancia entre $\hat{y}$ y $f(x)$.
- ▶ **Error hacia atrás**: menor perturbación Δx para la cual $\hat{y} = f(x + \Delta x)$.
- ▶ **Residuo**: defecto al sustituir $\hat{y}$ en la ecuación.

Lectura práctica

Un buen error hacia atrás significa que resolvimos exactamente un problema cercano al original.

El análisis diferencial aproxima el error

Sea $y = f(x)$ y suponga que el dato disponible es $\tilde{x} = x + \Delta x$. Entonces

$$\Delta y = f(\tilde{x}) - f(x) \approx f'(x) \Delta x.$$

Por tanto, en primer orden,

$$|\Delta y| \lesssim |f'(x)| |\Delta x|.$$

Análisis diferencial de error

Reemplaza localmente una función por su aproximación lineal para estimar cómo los errores de entrada afectan el resultado.

La aproximación mejora cuando Δx es pequeño y f es suave cerca de x .

Cada entrada aporta al error total

Si $y = f(x_1, \dots, x_n)$, la aproximación diferencial es

$$\Delta y \approx \sum_{j=1}^n \frac{\partial f}{\partial x_j}(x) \Delta x_j.$$

Cuando $y \neq 0$ y $x_j \neq 0$, también puede escribirse

$$\varepsilon_y \approx \sum_{j=1}^n \underbrace{\left(\frac{x_j}{y} \frac{\partial f}{\partial x_j}(x) \right)}_{\text{factor de amplificación}} \varepsilon_{x_j}.$$

Los signos permiten compensaciones; una cota conservadora usa valores absolutos.

Cobb-Douglas propaga errores de entrada

Para una función Cobb-Douglas

$$Y = AK^{\alpha}L^{\beta},$$

el análisis diferencial relativo produce

$$\varepsilon_Y \approx \varepsilon_A + \alpha\varepsilon_K + \beta\varepsilon_L.$$

Lectura cuantitativa

Si $\alpha = 0.35$, $\beta = 0.65$, A no cambia, K tiene un error de 1 % y L uno de 2 %, una cota de primer orden es

$$|\varepsilon_Y| \lesssim 0.35(1\%) + 0.65(2\%) = 1.65\%.$$

La fórmula identifica qué dato domina la incertidumbre del resultado.

El condicionamiento pertenece al problema

Una medida local de sensibilidad relativa para $y = f(x)$ es

$$\kappa_f(x) = \left| \frac{xf'(x)}{f(x)} \right|, \quad f(x) \neq 0.$$

Para perturbaciones pequeñas,

$$\frac{|\Delta y|}{|y|} \approx \kappa_f(x) \frac{|\Delta x|}{|x|}.$$

- ▶ κ pequeño: el problema amortigua o transmite moderadamente el error.
- ▶ κ grande: pequeñas perturbaciones pueden amplificarse mucho.

La sensibilidad depende del punto

Para

$$f(x) = \frac{1}{1-x}, \quad \kappa_f(x) = \left| \frac{x}{1-x} \right|.$$

► En $x = 0.1$, $\kappa_f(x) \approx 0.11$.

► En $x = 0.99$, $\kappa_f(x) = 99$.

La misma fórmula puede estar bien o mal condicionada dependiendo de los datos.

Consecuencia

Ningún algoritmo puede recuperar información que los datos ya no determinan con precisión suficiente.

La cancelación elimina cifras

Considere

$$g(x) = \sqrt{x+1} - \sqrt{x}.$$

Para $x = 10^{16}$, ambos radicales se redondean aproximadamente a 10^8 y la resta directa puede producir cero.

El valor matemático no es cero

$$g(10^{16}) \approx 5 \times 10^{-9}.$$

El problema no está en la fórmula matemática, sino en evaluarla mediante una resta de cantidades casi iguales.

Reformular puede recuperar precisión

Racionalizando,

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

Evaluación directa

Resta dos números grandes y cercanos.

Evaluación reformulada

Evita la cancelación y conserva la escala del resultado.

Equivalencia algebraica no implica equivalencia numérica.

Primitivas para casos delicados

Para x pequeño:

$$e^x - 1 \quad \text{y} \quad \log(1 + x)$$

sufren cancelación si se evalúan literalmente.

Funciones especializadas de NumPy

`np.expm1(x)` , `np.log1p(x)` .

Con $x = 10^{-16}$, `np.exp(x)-1` puede dar 0, mientras `np.expm1(x)` conserva la escala 10^{-16} .

Fuente: documentación oficial de `numpy.expm1` y `numpy.log1p`.

Cada operación propaga de forma diferente

Para perturbaciones relativas pequeñas,

$$\varepsilon_{xy} \approx \varepsilon_x + \varepsilon_y, \quad \varepsilon_{x/y} \approx \varepsilon_x - \varepsilon_y,$$

$$\varepsilon_{\sqrt{x}} \approx \frac{1}{2}\varepsilon_x, \quad \varepsilon_{x \pm y} \approx \frac{x}{x \pm y}\varepsilon_x + \frac{y}{x \pm y}\varepsilon_y.$$

La resta es delicada

Si $x \approx y$, los factores $x/(x - y)$ y $y/(x - y)$ pueden ser enormes.

Entrada y redondeo recorren rutas distintas

Para calcular $y = f(x)$ recibimos $\tilde{x} = x + \Delta x$ y evaluamos con aritmética finita:

$$\hat{y} = \text{fl}(f(\tilde{x})).$$

En primer orden,

$$\hat{y} - f(x) \approx f'(x)\Delta x + \underbrace{\left(\text{fl}(f(\tilde{x})) - f(\tilde{x}) \right)}_{\text{efecto del algoritmo y el redondeo}}.$$

Condicionamiento y estabilidad analizan estos dos mecanismos por separado.

Un algoritmo encadena operaciones

Un algoritmo transforma datos de entrada en un resultado mediante un número finito de operaciones elementales.

$$x \longrightarrow z_1 \longrightarrow z_2 \longrightarrow \cdots \longrightarrow z_r \longrightarrow y.$$

- ▶ Los z_i son resultados intermedios y también se redondean.
- ▶ Dos algoritmos pueden calcular la misma función exacta con rutas distintas.
- ▶ Analizar la ruta permite localizar amplificación, cancelación y costo.

Un problema numérico define un mapeo

Si las entradas son $x_1, \dots, x_n$ y las salidas son $y_1, \dots, y_m$, reunimos los datos como

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

Resolver el problema significa evaluar

$$\varphi : D \subset \mathbb{R}^n \longrightarrow \mathbb{R}^m, \quad y = \varphi(x),$$

donde $y_i = \varphi_i(x_1, \dots, x_n)$.

El problema no fija el algoritmo

Una misma función φ puede evaluarse mediante secuencias distintas.

Los operandos cambian en cada paso

En la etapa i , el algoritmo dispone de un conjunto de operandos

$$x^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \vdots \\ x_{n_i}^{(i)} \end{bmatrix} \in \mathbb{R}^{n_i}.$$

Una operación elemental transforma ese conjunto:

$$\varphi^{(i)} : D_i \subset \mathbb{R}^{n_i} \longrightarrow \mathbb{R}^{n_{i+1}}, \quad x^{(i+1)} = \varphi^{(i)}(x^{(i)}).$$

Los operandos pueden incluir entradas originales y resultados intermedios.

El algoritmo es una composición

Si el algoritmo tiene $r + 1$ etapas, entonces

$$x = x^{(0)} \xrightarrow{\varphi^{(0)}} x^{(1)} \xrightarrow{\varphi^{(1)}} \dots \xrightarrow{\varphi^{(r)}} x^{(r+1)} = y.$$

Por tanto, la función evaluada por la ruta exacta es

$$\varphi = \varphi^{(r)} \circ \varphi^{(r-1)} \circ \dots \circ \varphi^{(0)}.$$

Dos niveles de análisis

La función φ determina la sensibilidad del problema; su descomposición en mapas elementales determina cómo el algoritmo introduce y propaga redondeos.

La suma tiene dos descomposiciones

Para $\varphi(a, b, c) = a + b + c$:

Ruta 1: sumar $a + b$

$$\varphi_1^{(0)}(a, b, c) = (a + b, c),$$

$$\varphi_1^{(1)}(u, v) = u + v.$$

Ruta 2: sumar $b + c$

$$\varphi_2^{(0)}(a, b, c) = (a, b + c),$$

$$\varphi_2^{(1)}(u, v) = u + v.$$

$$\varphi = \varphi_1^{(1)} \circ \varphi_1^{(0)} = \varphi_2^{(1)} \circ \varphi_2^{(0)}.$$

La igualdad exacta no implica el mismo comportamiento en punto flotante.

Una misma suma admite rutas diferentes

Para $y = a + b + c$ podemos usar

$$z_1 = a + b, \quad y = z_1 + c,$$

o bien

$$z_1 = b + c, \quad y = a + z_1.$$

En primer orden, los errores de redondeo relativos contienen factores como

$$\frac{a + b}{a + b + c} \quad \text{o} \quad \frac{b + c}{a + b + c}.$$

La ruta más conveniente depende de los valores concretos de los sumandos.

Cada paso combina dos fuentes de error

Defina el error acumulado antes de la etapa i como $\Delta x^{(i)} = \tilde{x}^{(i)} - x^{(i)}$ y el redondeo local como

$$\alpha_{i+1} = \text{fl}\left(\varphi^{(i)}(\tilde{x}^{(i)})\right) - \varphi^{(i)}(\tilde{x}^{(i)}).$$

Entonces

$$\Delta x^{(i+1)} \approx D\varphi^{(i)}(x^{(i)}) \Delta x^{(i)} + \alpha_{i+1}.$$

Lectura de la recurrencia

El primer término transporta el error ya existente; el segundo incorpora el nuevo redondeo de la etapa.

El redondeo local tiene un modelo relativo

Si cada componente calculada se redondea con error relativo acotado por la unidad de redondeo u , podemos escribir

$$\alpha_i = E_i x^{(i)}, \quad E_i = \text{diag}(\varepsilon_1^{(i)}, \dots, \varepsilon_{n_i}^{(i)}), \quad |\varepsilon_j^{(i)}| \leq u.$$

Equivalentemente,

$$\text{fl}(\varphi^{(i-1)}(v)) = (I + E_i)\varphi^{(i-1)}(v).$$

Qué representa E_i

No es un dato del problema: registra los redondeos introducidos en una etapa concreta del algoritmo.

Los errores viajan hasta la salida

El mapa remanente desde la etapa i es

$$\psi^{(i)} = \varphi^{(r)} \circ \varphi^{(r-1)} \circ \dots \circ \varphi^{(i)}.$$

Su derivada mide cuánto se amplifica un error introducido en $x^{(i)}$ antes de alcanzar la salida:

$$D\psi^{(i)}(x^{(i)}) = D\varphi^{(r)}(x^{(r)}) \dots D\varphi^{(i)}(x^{(i)}).$$

Regla de la cadena

Un redondeo temprano atraviesa más etapas; uno tardío recorre un camino más corto. Su efecto depende de las derivadas de todas las etapas restantes.

El error final suma las contribuciones

En primer orden, el error de salida satisface

$$\Delta y \approx \underbrace{D\varphi(x) \Delta x}_{\text{error de entrada}} + \sum_{i=1}^r \underbrace{D\psi^{(i)}(x^{(i)}) \alpha_i}_{\text{redondeo de la etapa } i} + \underbrace{\alpha_{r+1}}_{\text{redondeo final}}.$$

Como $\alpha_i = E_i x^{(i)}$, cada aporte depende de

- ▶ el tamaño del resultado intermedio;
- ▶ el redondeo local;
- ▶ la amplificación a través de las etapas restantes.

Esta separación permite comparar algoritmos que calculan la misma φ .

Condicionamiento y estabilidad son distintos

Problema condicionado

Describe cuánto cambia la solución exacta cuando cambian los datos.

Algoritmo estable

Produce el resultado exacto o casi exacto de un problema cercano.

error hacia adelante $\lesssim$ condicionamiento $\times$ error hacia atrás.

Un problema mal condicionado puede limitar incluso a un algoritmo estable.

Dos algoritmos, distinta estabilidad

Algoritmo A

$$s_1 = a^2, \quad s_2 = b^2, \quad y = s_1 - s_2.$$

Resta dos cuadrados cercanos cuando $a \approx b$.

Algoritmo B

$$d = a - b, \quad s = a + b, \quad y = d s.$$

Separa la diferencia pequeña antes de formar magnitudes grandes.

El laboratorio comparará ambos algoritmos con precisión de referencia.

Las dos rutas tienen mapas distintos

Algoritmo A

$$\varphi_A^{(0)}(a, b) = \begin{bmatrix} a^2 \\ b \end{bmatrix}, \quad \varphi_A^{(1)}(u, v) = \begin{bmatrix} u \\ v^2 \end{bmatrix},$$

$$\varphi_A^{(2)}(u, v) = u - v.$$

Algoritmo B

$$\varphi_B^{(0)}(a, b) = \begin{bmatrix} a + b \\ a - b \end{bmatrix},$$

$$\varphi_B^{(1)}(u, v) = uv.$$

Ambas composiciones producen $a^2 - b^2$ en aritmética exacta.

El diferencial separa entrada y redondeo

Para ambos algoritmos, el aporte de los datos es

$$2a \Delta a - 2b \Delta b.$$

Los aportes de redondeo sí dependen de la ruta:

$$\Delta y_A \approx 2a\Delta a - 2b\Delta b + a^2\varepsilon_1 - b^2\varepsilon_2 + (a^2 - b^2)\varepsilon_3,$$

$$\Delta y_B \approx 2a\Delta a - 2b\Delta b + (a^2 - b^2)(\varepsilon_1 + \varepsilon_2 + \varepsilon_3).$$

Consecuencia

El condicionamiento de $a^2 - b^2$ es común; la acumulación de redondeos cambia con la descomposición algorítmica.

La estabilidad depende de los datos

Para $y = a^2 - b^2$, las cotas de redondeo de primer orden son

$$E_A \lesssim (a^2 + b^2 + |a^2 - b^2|)u,$$

$$E_B \lesssim 3|a^2 - b^2|u,$$

donde A evalúa $a^2 - b^2$ directamente y B usa $(a - b)(a + b)$.

Conclusión del ejemplo original

B es más estable cuando $\frac{1}{3} < |a/b|^2 < 3$; fuera de esa zona la comparación puede favorecer a A. No existe una reformulación universalmente superior.

El orden de suma cambia el resultado

Para los datos 10^{16} , 1 y -10^{16} :

$$(10^{16} + 1) - 10^{16} = 0,$$

$$(10^{16} - 10^{16}) + 1 = 1.$$

- ▶ La suma secuencial acumula un redondeo en cada paso.
- ▶ La suma por pares puede mejorar la precisión en muchos casos.
- ▶ `math.fsum` prioriza precisión; `np.sum` equilibra precisión y rendimiento.

Fuente: documentación oficial de `numpy.sum`.

Exactitud y costo deben evaluarse juntos

Al comparar algoritmos debemos considerar

- ▶ número de operaciones aritméticas;
- ▶ memoria utilizada y movimiento de datos;
- ▶ posibilidad de vectorización o paralelización;
- ▶ error de aproximación y error de redondeo;
- ▶ robustez ante datos extremos.

Principio de diseño

El algoritmo preferible logra la precisión necesaria con un costo razonable; no necesariamente minimiza una sola de estas cantidades.

Vectorizar cambia la implementación

Bucle en Python

```
s = 0.0
for i in range(n):
    s += a[i]*b[i]
```

Operación NumPy

```
s = np.dot(a, b)
```

Ambas expresiones calculan un producto interno, pero utilizan capas de ejecución distintas. El laboratorio comparará tiempo, escala y reproducibilidad.

El orden de costo anticipa el crecimiento

Si n mide el tamaño del problema, expresamos el trabajo dominante mediante notación de orden:

$$O(n)$$

Recorrer o sumar un vector una vez.

$$O(n^2)$$

Comparar cada par entre n observaciones.

$$O(n^3)$$

Un procedimiento con tres recorridos anidados.

El tiempo medido depende del equipo; el orden describe cómo cambia al crecer n .

NumPy y SciPy se complementan

NumPy

Arreglos, tipos numéricos, operaciones vectorizadas y álgebra básica.

SciPy

Algoritmos científicos para ecuaciones, integración, interpolación y optimización.

Uso responsable

La biblioteca implementa el método; el usuario todavía debe revisar escalas, condicionamiento, tolerancias, residuo y mensajes de terminación.

Cinco preguntas para auditar

1. **Representación:** ¿qué precisión y rango tienen los datos?
2. **Escala:** ¿conviene error absoluto o relativo?
3. **Sensibilidad:** ¿qué predice el análisis diferencial?
4. **Algoritmo:** ¿evita cancelación y acumulación innecesaria?
5. **Validación:** ¿el residuo y una referencia confirman el resultado?

El laboratorio verifica las ideas

- ▶ Inspeccionar ϵ , rango, separación y números adyacentes.
- ▶ Comparar truncamiento y redondeo al aproximar una derivada.
- ▶ Reproducir no asociatividad, overflow, underflow y cancelación.
- ▶ Comparar fórmulas directas con `expm1` y `log1p`.
- ▶ Contrastar una predicción diferencial y medir errores y residuos.
- ▶ Descomponer y comparar sumas, productos internos y costos.

Cinco ideas que deben permanecer

1. Los números de máquina son finitos y las operaciones introducen redondeo.
2. Truncamiento y redondeo son errores distintos y pueden competir.
3. El análisis diferencial transporta perturbaciones a través de cada etapa.
4. El condicionamiento es una propiedad del problema; la estabilidad es una propiedad del algoritmo.
5. Reformular y validar puede ser tan importante como elegir el método.

Un resultado numérico confiable debe poder explicarse y auditarse.

Comprobación de salida

Sin ejecutar código, anticipe el resultado de

$$a = 10^{16}, \quad b = -10^{16}, \quad c = 1.$$

1. ¿Cuánto vale $(a + b) + c$ en punto flotante?
2. ¿Cuánto vale $a + (b + c)$?
3. ¿Qué concepto explica la diferencia?
4. ¿El problema está mal condicionado o el algoritmo perdió información?

Referencias

Textos

- ▶ Stoer y Bulirsch (2002). *Introduction to Numerical Analysis*.
- ▶ Higham (2002). *Accuracy and Stability of Numerical Algorithms*.
- ▶ Burden y Faires (2004). *Numerical Analysis*.

Documentación oficial

- ▶ `finfo`: precisión y rango.
- ▶ `expm1` y `log1p`.
- ▶ `sum`: precisión de la suma.